МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(ДГТУ)

Факультет «Информатика и вычислительная техника»

Кафедра «Математика и информатика»

ЛАБОРАТОРНАЯ РАБОТА №2

Тема: «КОМПОЗИЦИЯ И ПЕРЕГРУЗКА ОПЕРАТОРОВ»

Дисциплина (модуль) «Программирование»

Направление подготовки/специальность 09.03.02 Информационные системы и технологии

Направленность (профиль) Искусственный интеллект и математическое моделирование

Номер варианта 2 Группа ВИИМ21

Обучающаяся _____ Шевченко О.Н.
подпись, дата

Лабораторную работу проверил(а) _____ доц. Н. Д. Панасенко
подпись, дата

Ростов-на-Дону
2026

Цель работы: изучить и практически освоить принципы композиции объектов и перегрузки операторов в объектно-ориентированном программировании на Python, сформировать навыки проектирования классов с использованием отношений "has-a" (композиция) и создания интуитивно понятных интерфейсов через переопределение стандартных операторов, а также освоить визуальное моделирование разработанных программных сущностей с помощью UML.

Задание

1. Требуется написать программу и разработать интерфейс, которая вычисляет площадь для определенных работ в помещении. При этом есть части, в которых работы проводить не надо. Сохранять саму площадь помещения в поле не обязательно. Площади должны вычислять лишь при необходимости путем вызова методов. Программа должна запрашивать у пользователя данные и выдает ему площадь работ и количество необходимых материалов. Дано первоначальное описание, но его требуется исправить и доработать, согласно плану.

2. Напишите программу по следующему описанию.

Рекомендации:

− проверка типов: всегда проверяйте тип other в перегруженных операторах;

− обработка граничных значений: особенно важно для классов с ограниченным диапазоном (Brightness);

− округление: в методах __truediv__ используйте округление до целого, если требуется;

− форматирование вывода: следите за читаемостью выводимых строк;

− валидация данных: проверяйте корректность входных данных, особенно при вызове объекта.

3. Разработайте программу, моделирующую некоторую предметную область, в которой один объект (целое) содержит в себе другие объекты (части). Программа должна позволять:

− добавлять/удалять составные части.

− вычислять некоторую итоговую характеристику целого на основе характеристик частей (например, суммарную площадь, общий вес, время выполнения, точность и т.п.).

− выводить результат расчёта в удобном формате.

Требуется реализовать два класса (целое и часть) с использованием композиции. Для целого класса определить метод вычисления итоговой характеристики и метод, возвращающий количество необходимых ресурсов (материалов, времени, и т.п.) на основе расчёта. Разработать консольный интерфейс для ввода данных пользователем и вывода результата.

4. Разработайте класс, моделирующий некоторую величину (физическую, экономическую, информационную) с поддержкой арифметических операций. Класс должен содержать:

− конструктор, инициализирующий значение величины (целое или вещественное, по варианту);

− перегруженные операторы: +, -, *, / (обычное деление, при необходимости с округлением до целого);

− метод преобразования в другие единицы измерения (если применимо) или метод, вычисляющий производную характеристику (например, время чтения, плотность, мощность);

− возможность вызова экземпляра как функции для перезаписи значения.

Требуется реализовать корректную обработку типов (other должен быть числом или другим экземпляром того же класса), а также контроль диапазона значений (если необходимо). Разработать консольный интерфейс для ввода данных пользователем и вывода результата.

Ход работы

1. Расчёт напольного покрытия

Требуется вычислить площадь укладки напольного покрытия, исключив колонны и выступы. У объектов Floor должны быть поля length, width и obstacles. Площади вычисляются по вызову методов, а не сохраняются в полях. Метод tiles_count принимает размеры одной плитки и возвращает необходимое количество плиток. Разработан консольный интерфейс (см. рисунки 1, 2).

Я реализовала композицию Floor и Obstacle: пол создаёт и хранит препятствия. Полная площадь равна length · width, рабочая площадь равна полной площади за вычетом суммарной площади препятствий. Количество плиток вычисляется как ceil(Sраб / (a · b)). Предполагается, что препятствия не пересекаются и расположены внутри пола; запас на подрезку и ширина швов не учитываются. Проверяются положительность размеров и допустимость суммарной площади исключений.

Контрольный пример: пол 6 × 5 м; препятствия 0,5 × 0,5 м и 1 × 0,5 м; плитка 0,5 × 0,5 м. Получено: Sраб = 30 − 0,25 − 0,5 = 29,25 м²; N = ceil(29,25 / 0,25) = 117 шт.

```
Расчёт напольного покрытия (размеры в метрах)
Препятствия не пересекаются; расчёт без запаса.
Длина пола: 6
Ширина пола: 5
Количество препятствий: 2
Препятствие 1
Ширина: 0.5
Длина: 0.5
Препятствие 2
Ширина: 1
Длина: 0.5
Ширина плитки: 0.5
Длина плитки: 0.5
Полная площадь пола: 30.00 м²
Площадь укладки: 29.25 м²
Количество плиток: 117 шт.
```

Рисунок 1 – Результат работы программы расчёта напольного покрытия

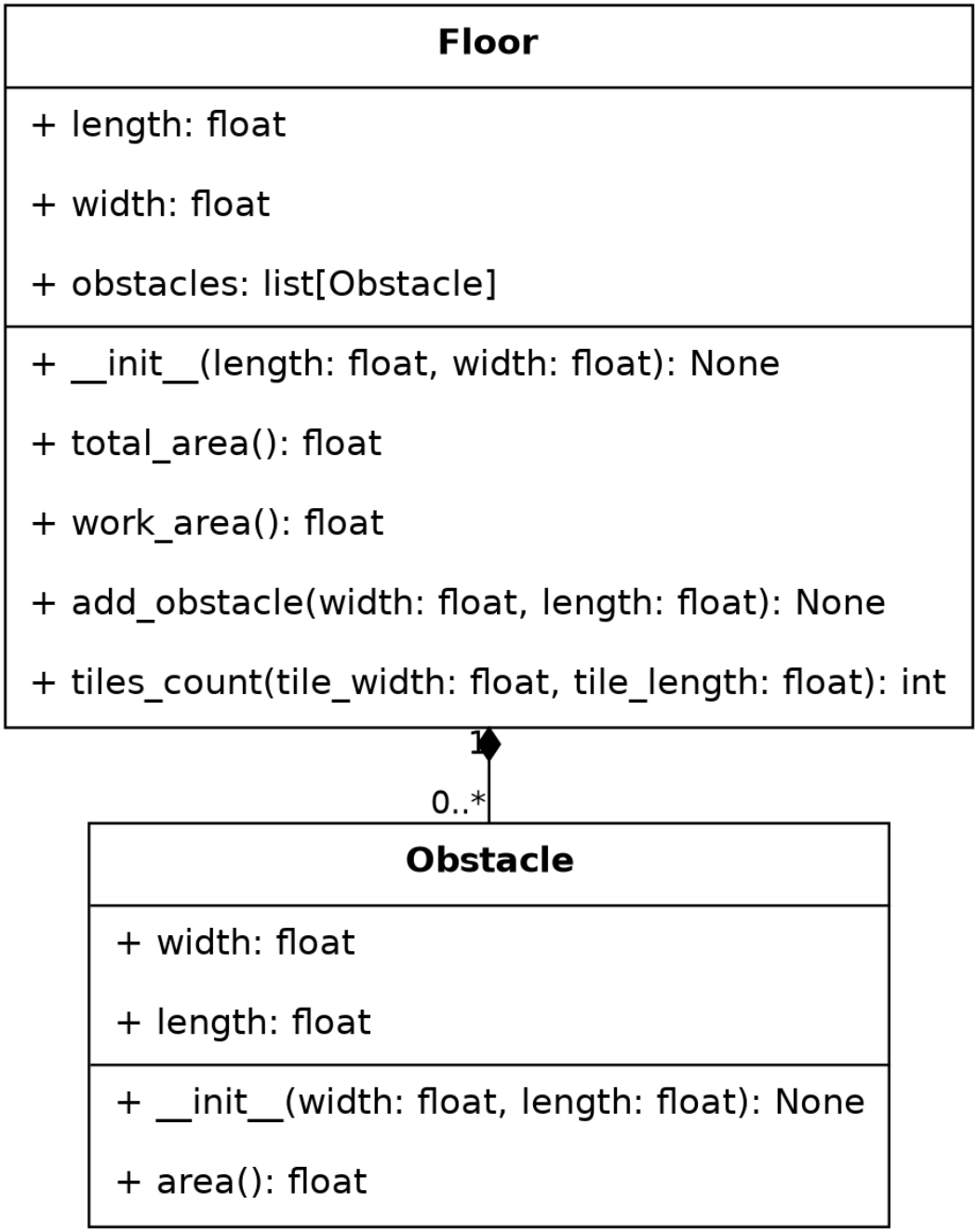


Рисунок 2 – UML-диаграмма классов Floor и Obstacle

Листинг 1 – Код программы

```
"""Задание 1, вариант 2. Расчёт напольного покрытия."""
from math import ceil, isfinite


def positive(value):
    if (type(value) not in (int, float)
            or not isfinite(value) or value <= 0):
        raise ValueError("Размер должен быть числом > 0")
    return float(value)
```

```
class Obstacle:
    def __init__(self, width, length):
        self.width = positive(width)
        self.length = positive(length)

    def area(self):
        return self.width * self.length


class Floor:
    def __init__(self, length, width):
        self.length = positive(length)
        self.width = positive(width)
        self.obstacles = []

    def total_area(self):
        return self.length * self.width

    def work_area(self):
        return self.total_area() - sum(
            obstacle.area() for obstacle in self.obstacles)

    def add_obstacle(self, width, length):
        obstacle = Obstacle(width, length)
        if not ((width <= self.width and length <= self.length)
                or (length <= self.width
                    and width <= self.length)):
            raise ValueError("Препятствие не помещается на полу")
        if obstacle.area() > self.work_area():
            raise ValueError("Препятствия больше площади пола")
        self.obstacles.append(obstacle)

    def tiles_count(self, tile_width, tile_length):
        tile_area = positive(tile_width) * positive(tile_length)
        return ceil(self.work_area() / tile_area)


def main():
    print("Расчёт напольного покрытия (размеры в метрах)")
    print("Препятствия не пересекаются; расчёт без запаса.")
    floor = Floor(float(input("Длина пола: ")),
                  float(input("Ширина пола: ")))
    count = int(input("Количество препятствий: "))
    if count < 0:
        raise ValueError("Количество не может быть отрицательным")
    for i in range(count):
        print(f"Препятствие {i + 1}")
        floor.add_obstacle(float(input("Ширина: ")),
                           float(input("Длина: ")))
    width = float(input("Ширина плитки: "))
    length = float(input("Длина плитки: "))
    print(f"Полная площадь пола: {floor.total_area():.2f} м²")
```

```
    print(f"Площадь укладки: {floor.work_area():.2f} м²")
    count = floor.tiles_count(width, length)
    print(f"Количество плиток: {count} шт.")


if __name__ == "__main__":
    try:
        main()
    except (ValueError, TypeError, OverflowError) as error:
        print(f"Ошибка: {error}")
```

2. Класс Book (Книга)

Требуется создать класс Book, содержащий целое количество страниц. Реализовать перегрузку операторов +, −, *, /; при делении округлять результат до целого. Метод reading_time принимает скорость чтения в страницах в час и возвращает строку с длительностью чтения. Вызов экземпляра как функции перезаписывает количество страниц (см. рисунки 3, 4).

Я реализовала проверку операндов: поддерживаются Book и конечные числа int/float, логические значения исключены. Число страниц неотрицательно и целочисленно. Для сложения, вычитания и умножения нецелый результат отклоняется; при делении применяется стандартный round Python (при равном удалении — к чётному целому). Отрицательный результат и деление на ноль запрещены. Неподдерживаемый тип приводит к NotImplemented и далее к TypeError. Исходные объекты при арифметических операциях не изменяются.

Контрольный пример: 300 и 120 страниц; сложение — 420, вычитание — 180, умножение первой книги на 2 — 600, деление на 2 — 150. При скорости 30 страниц/час чтение 300 страниц занимает 10 часов. Вызов first(450) устанавливает 450 страниц.

```
Класс Book
Страниц в первой книге: 300
Страниц во второй книге: 120
Скорость чтения (страниц/час): 30
Для прочтения 300 страниц потребуется 10 часов
Сложение: 420 страниц
Вычитание: 180 страниц
Умножение на 2: 600 страниц
Деление на 2: 150 страниц
Новое количество страниц первой книги: 450
После вызова объекта: 450 страниц
```

Рисунок 3 – Результат работы программы с классом Book

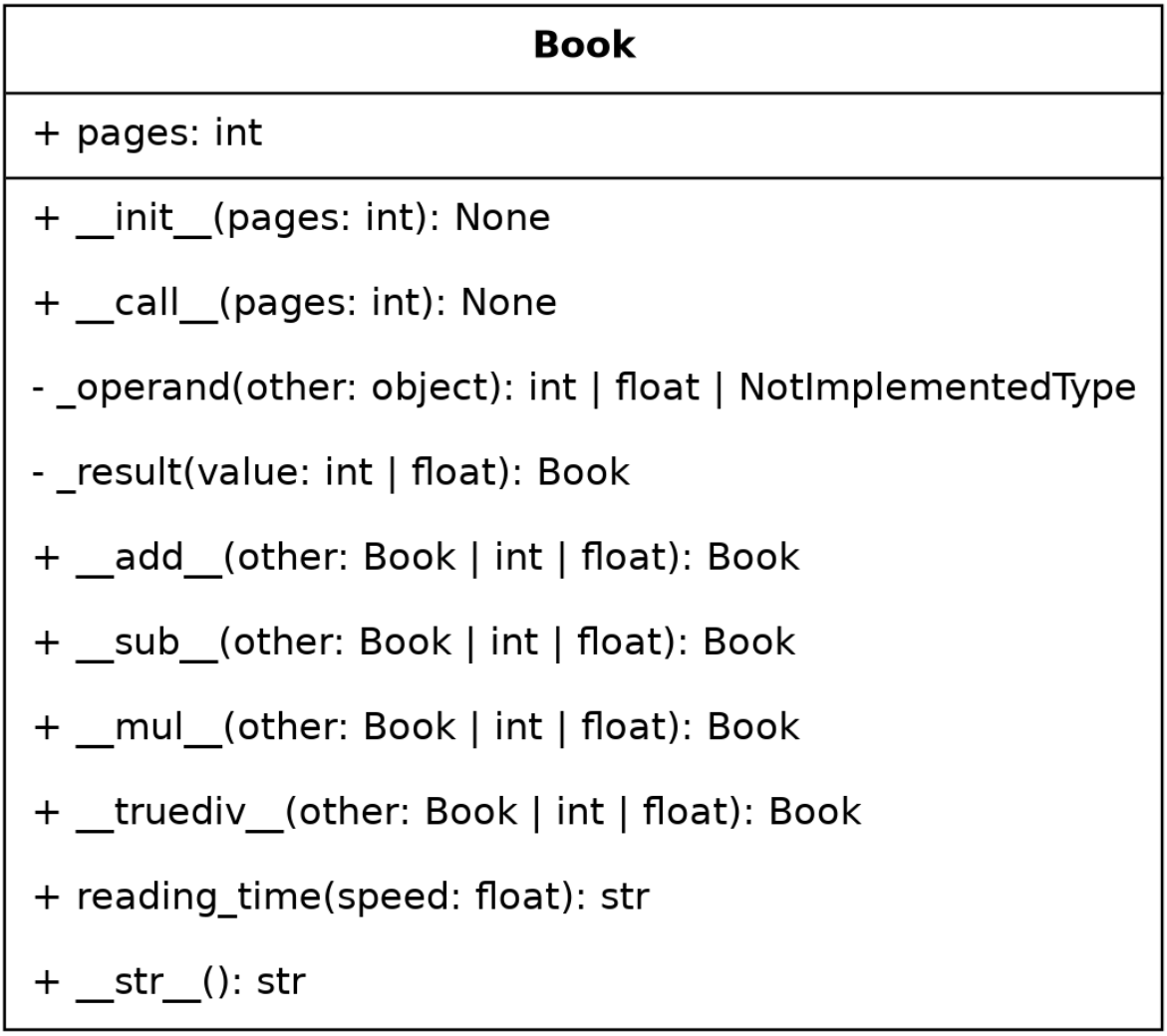


Рисунок 4 – UML-диаграмма класса Book

Листинг 2 – Код программы

```
"""Задание 2, вариант 2. Класс Book."""
from math import isfinite


class Book:
    def __init__(self, pages):
        self(pages)

    def __call__(self, pages):
        if type(pages) is not int:
            raise TypeError("Число страниц должно быть целым")
        if pages < 0:
            raise ValueError("Число страниц не может быть < 0")
        self.pages = pages

    @staticmethod
    def _operand(other):
        if isinstance(other, Book):
            return other.pages
        if type(other) in (int, float) and isfinite(other):
            return other
        return NotImplemented

    @staticmethod
    def _result(value):
        if not isfinite(value) or value < 0:
            raise ValueError("Недопустимое количество страниц")
        if value != int(value):
            raise ValueError("Результат должен быть целым числом")
        return Book(int(value))

    def __add__(self, other):
        value = self._operand(other)
        if value is NotImplemented:
            return NotImplemented
        return self._result(self.pages + value)

    def __sub__(self, other):
        value = self._operand(other)
        if value is NotImplemented:
            return NotImplemented
        return self._result(self.pages - value)

    def __mul__(self, other):
        value = self._operand(other)
        if value is NotImplemented:
            return NotImplemented
        return self._result(self.pages * value)
```

```
    def __truediv__(self, other):
        value = self._operand(other)
        if value is NotImplemented:
            return NotImplemented
        if value == 0:
            raise ZeroDivisionError("Деление на ноль")
        result = self.pages / value
        if result < 0:
            raise ValueError("Результат должен быть >= 0")
        return Book(round(result))

    def reading_time(self, speed):
        if (type(speed) not in (int, float)
                or not isfinite(speed) or speed <= 0):
            raise ValueError("Скорость чтения должна быть > 0")
        hours = self.pages / speed
        return (f"Для прочтения {self.pages} страниц "
                f"потребуется {hours:g} часов")

    def __str__(self):
        return f"{self.pages} страниц"


def main():
    print("Класс Book")
    first = Book(int(input("Страниц в первой книге: ")))
    second = Book(int(input("Страниц во второй книге: ")))
    speed = float(input("Скорость чтения (страниц/час): "))
    print(first.reading_time(speed))
    for name, operation in (
            ("Сложение", lambda: first + second),
            ("Вычитание", lambda: first - second),
            ("Умножение на 2", lambda: first * 2),
            ("Деление на 2", lambda: first / 2)):
        try:
            print(f"{name}: {operation()}")
        except (ValueError, ZeroDivisionError) as error:
            print(f"{name}: ошибка — {error}")
    first(int(input("Новое количество страниц первой книги: ")))
    print("После вызова объекта:", first)


if __name__ == "__main__":
    try:
        main()
    except (ValueError, TypeError, OverflowError) as error:
        print(f"Ошибка: {error}")
```

3. Ансамбль моделей (Ensemble)

Вариант 2 для профиля «Искусственный интеллект и математическое моделирование»: ансамбль состоит из базовых моделей Model. У модели заданы точность и время обучения. Требуется добавлять и удалять модели, вычислять среднюю точность и общее время обучения, определять оптимальное количество моделей для заданной точности. Разработаны консольный интерфейс, диаграмма классов и диаграмма последовательности (см. рисунки 5–7).

Я реализовала композицию: метод add_model создаёт Model внутри Ensemble, а remove_model удаляет её по индексу. Точность задаётся в долях единицы [0; 1], время обучения — в минутах и не может быть отрицательным. Средняя точность равна сумме точностей, делённой на число моделей. Время последовательного обучения равно сумме времён отдельных моделей. Для пустого ансамбля обе итоговые характеристики равны нулю.

Критерий оптимальности: минимальное число моделей в непустом подмножестве со средней точностью не ниже целевой. Модели упорядочиваются по убыванию точности; проверяются средние значения префиксов. При таком критерии достаточно одной лучшей модели, если её точность достигает цели; иначе подходящего подмножества нет. Это следствие того, что среднее не может превышать максимальный элемент. Рост фактической точности совместных предсказаний здесь не моделируется: в задании не заданы прогнозы моделей или правило голосования.

Контрольный пример: точности 0,80; 0,90; 0,85 и времена 10; 20; 15 минут. Средняя точность — 85 %, общее время — 45 минут. Для цели 0,88 достаточно одной модели с точностью 0,90. После удаления второй модели средняя точность равна 82,5 %, время — 25 минут, цель 0,88 недостижима. На рисунке 5 повторяющиеся строки меню опущены; полный протокол запуска приложен к исходникам.

```
Ансамбль моделей
Действие: 1
Точность (0–1): 0.8
Время обучения (мин): 10
Действие: 1
Точность (0–1): 0.9
Время обучения (мин): 20
Действие: 1
Точность (0–1): 0.85
Время обучения (мин): 15
Действие: 3
1. Точность: 80.00%; обучение: 10 мин
2. Точность: 90.00%; обучение: 20 мин
3. Точность: 85.00%; обучение: 15 мин
Средняя точность: 85.00%
Общее время обучения: 45 мин
Целевая точность (0–1): 0.88
Минимум моделей: 1
Действие: 2
Номер модели: 2
Модель удалена
Действие: 3
1. Точность: 80.00%; обучение: 10 мин
2. Точность: 85.00%; обучение: 15 мин
Средняя точность: 82.50%
Общее время обучения: 25 мин
Целевая точность (0–1): 0.88
Минимум моделей: цель недостижима
Действие: 0
```

Рисунок 5 – Результат работы программы с ансамблем моделей

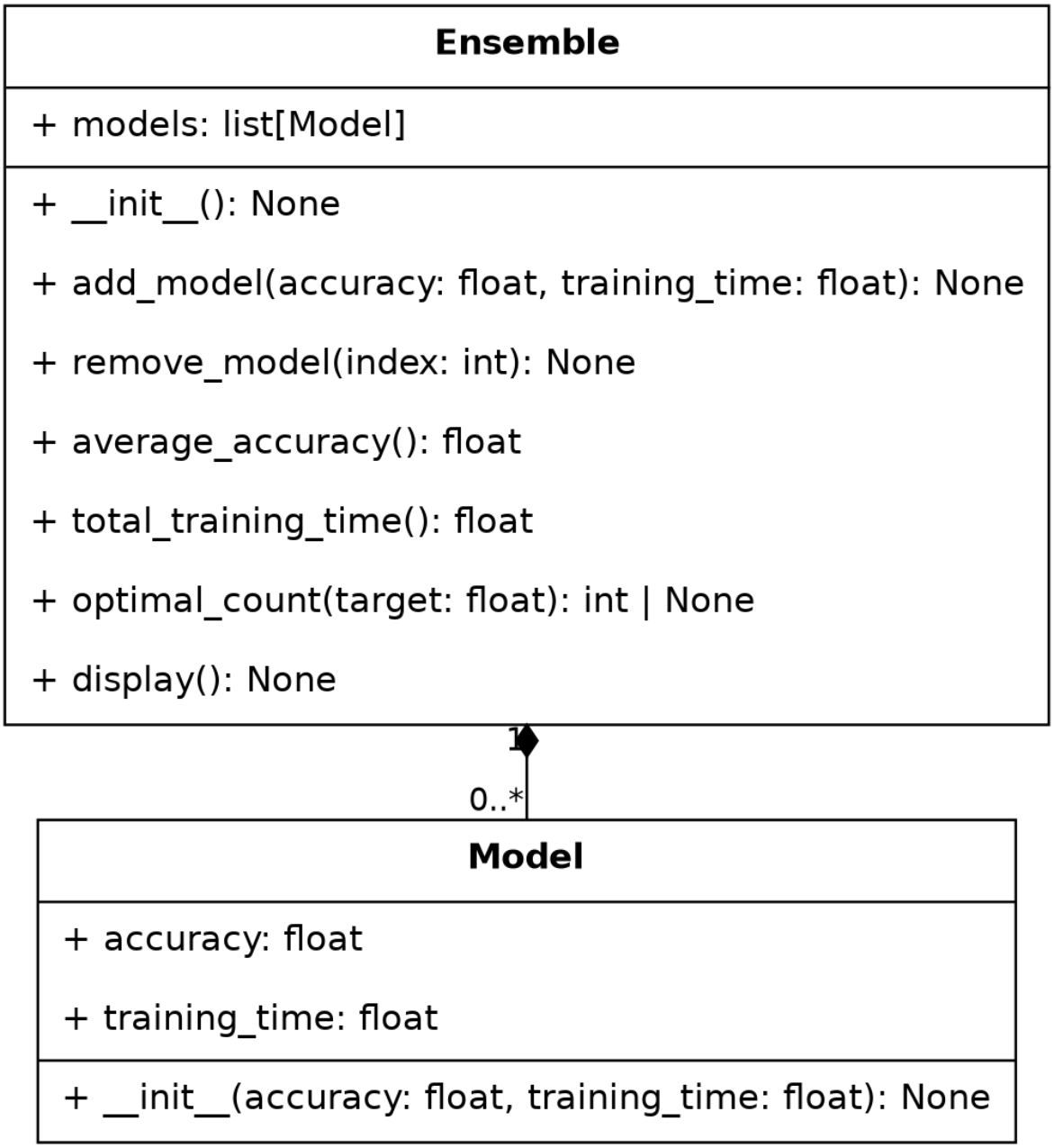


Рисунок 6 – UML-диаграмма классов Ensemble и Model

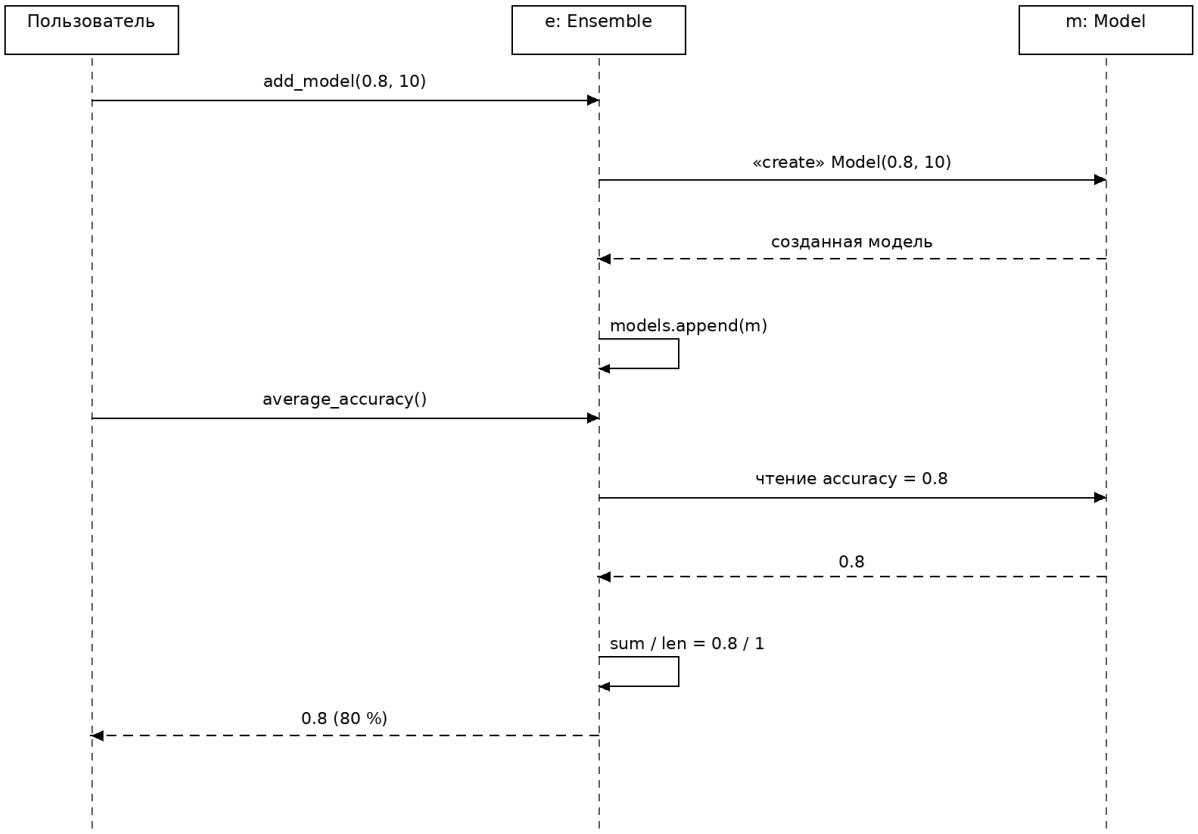


Рисунок 7 – Диаграмма последовательности добавления модели и расчёта точности

Листинг 3 – Код программы

```
"""Задание 3, вариант 2. Ансамбль моделей."""
from math import isfinite


def number(value, minimum, maximum=float("inf")):
    if (type(value) not in (int, float) or not isfinite(value)
            or not minimum <= value <= maximum):
        raise ValueError("Число вне допустимого диапазона")
    return float(value)


class Model:
    def __init__(self, accuracy, training_time):
        self.accuracy = number(accuracy, 0, 1)
        self.training_time = number(training_time, 0)


class Ensemble:
    def __init__(self):
        self.models = []
```

```
    def add_model(self, accuracy, training_time):
        self.models.append(Model(accuracy, training_time))

    def remove_model(self, index):
        if (type(index) is not int
                or not 0 <= index < len(self.models)):
            raise IndexError("Неверный индекс модели")
        del self.models[index]

    def average_accuracy(self):
        if not self.models:
            return 0.0
        return (sum(m.accuracy for m in self.models)
                / len(self.models))

    def total_training_time(self):
        return sum(m.training_time for m in self.models)

    def optimal_count(self, target):
        target = number(target, 0, 1)
        # Критерий: минимум моделей, средняя точность >= target.
        # Лучшее среднее для k моделей дают k самых точных.
        ordered = sorted(self.models,
                         key=lambda m: m.accuracy, reverse=True)
        total = 0.0
        for count, model in enumerate(ordered, start=1):
            total += model.accuracy
            if total / count >= target:
                return count
        return None

    def display(self):
        if not self.models:
            print("Ансамбль пуст")
        for i, model in enumerate(self.models, start=1):
            print(f"{i}. Точность: {model.accuracy:.2%}; "
                  f"обучение: {model.training_time:g} мин")
        print(f"Средняя точность: {self.average_accuracy():.2%}")
        print(f"Общее время обучения: "
              f"{self.total_training_time():g} мин")


def main():
    ensemble = Ensemble()
    print("Ансамбль моделей")
    while True:
        print("1 — добавить; 2 — удалить; 3 — расчёт; 0 — выход")
        choice = input("Действие: ")
        try:
            if choice == "1":
                ensemble.add_model(
                    float(input("Точность (0–1): ")),
                    float(input("Время обучения (мин): ")))
```

```
            elif choice == "2":
                index = int(input("Номер модели: ")) - 1
                ensemble.remove_model(index)
                print("Модель удалена")
            elif choice == "3":
                ensemble.display()
                target = float(input("Целевая точность (0–1): "))
                count = ensemble.optimal_count(target)
                print("Минимум моделей:",
                      count if count is not None
                      else "цель недостижима")
            elif choice == "0":
                break
            else:
                print("Неизвестное действие")
        except (ValueError, TypeError, IndexError) as error:
            print(f"Ошибка: {error}")


if __name__ == "__main__":
    main()
```

4. Класс Loss (Потери)

Вариант 2 для профиля «Искусственный интеллект и математическое моделирование»: класс Loss хранит вещественное значение функции потерь, не меньшее нуля. Требуется реализовать +, −, *, /, проверку уменьшения потерь и обновление значения через вызов экземпляра как функции. Построены диаграмма класса и диаграмма последовательности сложения (см. рисунки 8–10).

Я реализовала операции с Loss и конечными числами int/float. Результат каждой операции — новый объект Loss; отрицательные потери, бесконечность, NaN, логические значения и деление на ноль не допускаются. Метод is_converging(previous) возвращает True, когда текущее значение строго меньше предыдущего. Это проверка уменьшения за один шаг, а не доказательство математической сходимости последовательности. Метод __call__ проверяет новое значение и только затем обновляет поле value.

Контрольный пример: для 0,8 и 0,2 получены сумма 1, разность 0,6, произведение 0,16 и частное 4. После вызова first(0.1) значение становится 0,1;

относительно 0,2 потери уменьшились.

```
Класс Loss
Первое значение потерь: 0.8
Второе значение потерь: 0.2
Сложение: 1
Вычитание: 0.6
Умножение: 0.16
Деление: 4
Новое значение первых потерь: 0.1
После вызова объекта: 0.1
Потери уменьшились относительно второго значения: да
```

Рисунок 8 – Результат работы программы с классом Loss

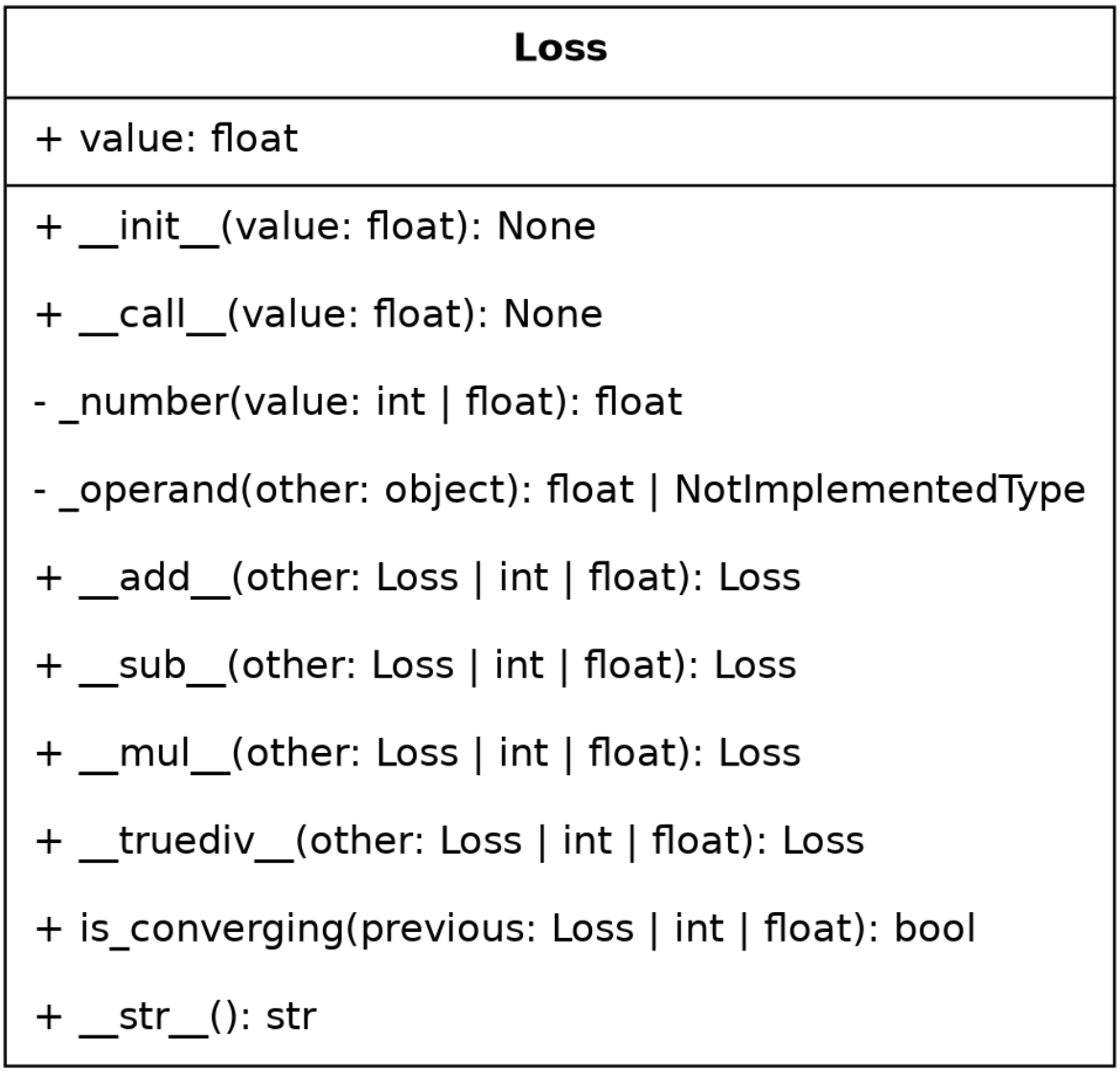


Рисунок 9 – UML-диаграмма класса Loss

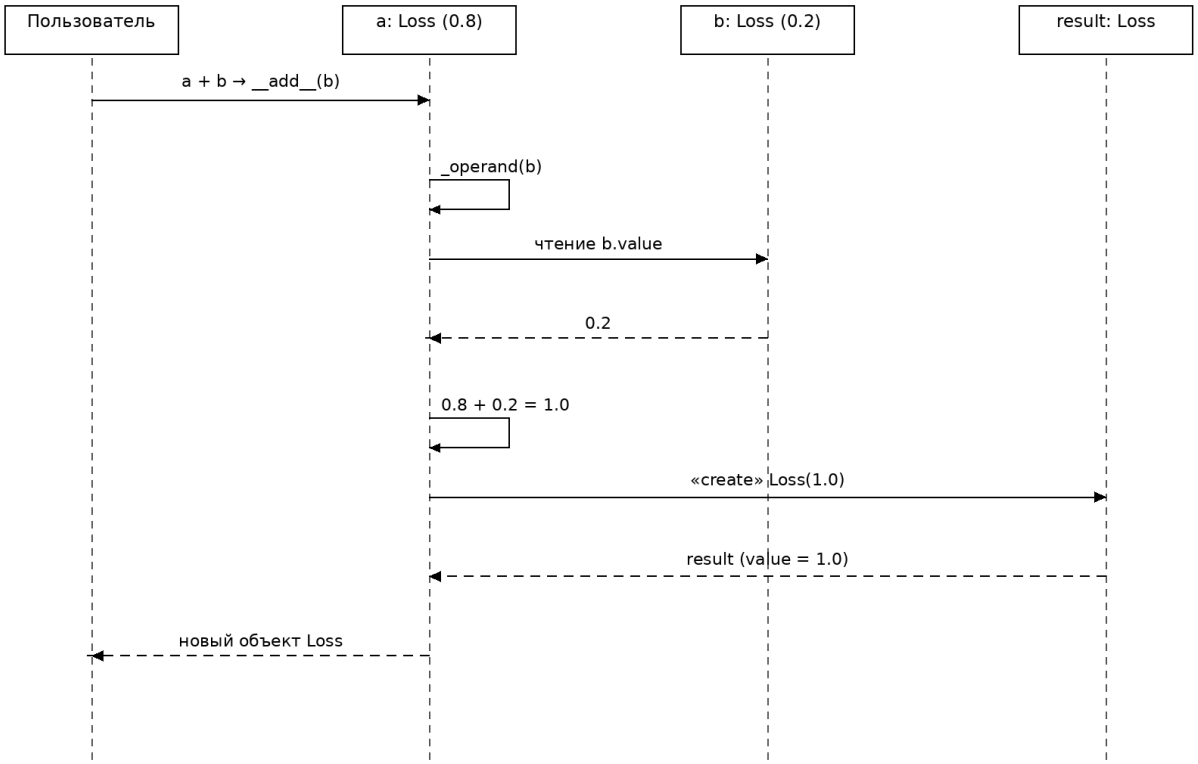


Рисунок 10 – Диаграмма последовательности сложения объектов Loss

Листинг 4 – Код программы

```
"""Задание 4, вариант 2. Класс Loss."""
from math import isfinite


class Loss:
    def __init__(self, value):
        self(value)

    @staticmethod
    def _number(value):
        if type(value) not in (int, float):
            raise TypeError("Значение должно быть числом")
        if not isfinite(value):
            raise ValueError("Число должно быть конечным")
        return float(value)

    def __call__(self, value):
        value = self._number(value)
        if value < 0:
            raise ValueError("Потери должны быть >= 0")
        self.value = value

    @staticmethod
    def _operand(other):
        if isinstance(other, Loss):
            return other.value
```

```
        if type(other) in (int, float):
            return Loss._number(other)
        return NotImplemented

    def __add__(self, other):
        value = self._operand(other)
        if value is NotImplemented:
            return NotImplemented
        return Loss(self.value + value)

    def __sub__(self, other):
        value = self._operand(other)
        if value is NotImplemented:
            return NotImplemented
        return Loss(self.value - value)

    def __mul__(self, other):
        value = self._operand(other)
        if value is NotImplemented:
            return NotImplemented
        return Loss(self.value * value)

    def __truediv__(self, other):
        value = self._operand(other)
        if value is NotImplemented:
            return NotImplemented
        if value == 0:
            raise ZeroDivisionError("Деление на ноль")
        return Loss(self.value / value)

    def is_converging(self, previous):
        value = self._operand(previous)
        if value is NotImplemented:
            raise TypeError("Ожидается Loss или число")
        if value < 0:
            raise ValueError("Предыдущие потери должны быть >= 0")
        return self.value < value

    def __str__(self):
        return f"{self.value:g}"


def main():
    print("Класс Loss")
    first = Loss(float(input("Первое значение потерь: ")))
    second = Loss(float(input("Второе значение потерь: ")))
    for name, operation in (
            ("Сложение", lambda: first + second),
            ("Вычитание", lambda: first - second),
            ("Умножение", lambda: first * second),
            ("Деление", lambda: first / second)):
        try:
            print(f"{name}: {operation()}")
```

```
        except (ValueError, ZeroDivisionError) as error:
            print(f"{name}: ошибка — {error}")
    first(float(input("Новое значение первых потерь: ")))
    print("После вызова объекта:", first)
    print("Потери уменьшились относительно второго значения:",
          "да" if first.is_converging(second) else "нет")


if __name__ == "__main__":
    try:
        main()
    except (ValueError, TypeError, OverflowError) as error:
        print(f"Ошибка: {error}")
```

Проверка результатов

Я проверила программы на Python 3 с помощью unittest: выполнены 15 тестов, все завершились успешно. Проверены расчёт площади и округление числа плиток вверх, пересчёт площади при изменении размеров, полностью исключённая площадь, арифметика Book и Loss с числами и экземплярами классов, время чтения, обновление значений, расчёты ансамбля до и после удаления модели, достижимая и недостижимая точность, пустой ансамбль. Проверены также некорректные типы, выход за допустимые диапазоны и деление на ноль. Для воспроизведения: python3 -m unittest discover -s programs -v из каталога комплекта работы.

Вывод: я изучила и практически освоила принципы композиции объектов и перегрузки операторов в объектно-ориентированном программировании на Python. Реализовала расчёт напольного покрытия, класс Book, ансамбль моделей и класс Loss для второго варианта. Сформировала навыки проектирования классов с отношением «has-a», проверки типов и граничных значений, создания консольных интерфейсов. Освоила визуальное моделирование программных сущностей с помощью UML-диаграмм классов и последовательности. Корректность основных расчётов подтвердила контрольными примерами и автоматическими тестами.